`Chris Kane-Pardy` & Niklas Anderson
Mugisha
5/15/2026

Wishbone Peripheral Integration

To integrate the VGA controller into the existing RVfpga system, we modified both the Wishbone interconnect and the VeeRwolf core similarly to Project 1. A new VGA peripheral was added at address 0x80001500, which required updates to both wb_intercon.v and wb_intercon.vh. Additional Wishbone signals were added for the new peripheral, including address, data, select, acknowledge, and control signals.

The VGA peripheral itself was implemented in a new module named wb_vga.sv. This module instantiated both the provided dtg.sv timing generator and the generated font ROM. The dtg module was responsible for generating VGA timing signals, including horizontal synchronization, vertical synchronization, active video timing, and the current pixel row and column coordinates.

Within the wb_vga module, a set of memory-mapped registers was implemented to support software control of the display. These included:

- A mode register for switching between graphics and text modes
- A coordinate register for selecting the display position
- A color data register for foreground and background color values
- A character register for storing ASCII character values

The coordinate register stored the display row in bits [19:10] and the display column in bits [9:0]. The color register stored separate foreground and background RGB values using 4-bit channels for each color component.

The VGA peripheral operated continuously by comparing the current pixel coordinates generated by the timing generator against the coordinates stored in the control registers. Depending on the current mode and register contents, the module selected the appropriate pixel color and drove the VGA synchronization and RGB outputs accordingly.

Text versus Graphics Mode

We chose to add a dedicated mode register to support switching between graphics and text modes. This allowed the development of both the text-based application and the screensaver-style graphics application while using a

single hardware bitstream. This also simplified testing because mode switching could occur entirely through software without requiring FPGA reprogramming.

Graphics mode and text mode shared most of the same hardware path. In graphics mode, the VGA peripheral rendered a colored 32x32 box at the programmed coordinate location. In text mode, the same coordinate registers were reused to position a rendered character on the display.

Adding a dedicated character register allowed the existing data register to be repurposed entirely for color storage. This simplified the interface between software and hardware and made the register layout more intuitive from the software side.

Top-Level VGA Signal Routing

Additional modifications were required to expose the VGA signals to the FPGA's external VGA connector. VGA synchronization and color outputs were added to both rvfpganexys.sv and veerwolf_core.sv. The signals were then connected to the Nexys A7 VGA interface through the project constraints file.

The design used four bits per color channel for red, green, and blue, along with separate horizontal and vertical synchronization signals. The VGA timing generator was configured for a standard 640x480 display mode.

Clock Modifications

The original RVfpga design operated using a 12.5 MHz system clock. However, VGA timing required a higher pixel frequency to generate a stable display signal. The system clock parameter was therefore modified to use a 25 MHz clock instead. The change was implemented by updating the clk_gen_nexys module's PLLE2_BASE instance to use a clock divider value of 64 rather than 128, producing a clock speed that was twice as fast as the default.

This modification introduced Vivado timing warnings during synthesis and implementation, but the generated bitstream still functioned correctly during hardware testing and successfully produced a stable VGA display output.

BRAM Configuration and Instantiation

To make use of the provided bitfont.coe file, we followed instructions from an MIT course for configuring BRAM within the Vivado GUI:

https://web.mit.edu/6.111/volume2/www/f2019/handouts/labs/lab3_19/rom_vivado.html

Because the font data was read-only, we configured the memory as a Single Port ROM using Vivado's Block Memory Generator IP. The generated module was named font_rom and configured with a width of 16 bits and a depth of 2048 entries. The width was selected because each character glyph consisted of a 16x16 bitmap, allowing each ROM access to return one row of pixel data for the currently selected character.

The ROM contents were initialized using the provided bigfont.coe file through the "Load Init File" option within the IP configuration menu. After generating the ROM IP, we retrieved the generated module port list from the .veo file located at <project root>/<project name>.gen/sources_1/ip/font_rom.

During testing, we discovered that the provided font ROM omitted the first 32 non-printable ASCII characters. Rather than requiring software to compensate for this offset, we chose to correct the indexing directly in hardware. Incoming ASCII values written by software were automatically offset before generating the ROM address, allowing the C applications to use standard ASCII values directly without additional software-side logic.

Application 1

Application 1 was implemented as a software-driven text rendering demonstration using the custom VGA peripheral. The application was written entirely in C and interacted with the hardware exclusively through memory-mapped register writes.

The software-defined volatile pointers corresponding to the VGA peripheral registers:

- Mode register
- Coordinate register
- Color register
- Character register

These pointers allowed the application to directly control the VGA hardware using ordinary C assignments. During initialization, the program configured the VGA peripheral for text mode, selected a fixed screen coordinate for rendering, and initialized the display colors.

Foreground and background colors were configured by packing multiple RGB values into a single 32-bit register value using helper macros defined in

software. This simplified the application code and made it easier to experiment with different display colors during testing. The final version used a white foreground character with a red background in order to satisfy the project requirement while maintaining strong visual contrast.

The application logic itself was implemented as a simple polling loop. The program iterated through decimal digits and generated ASCII characters directly in software by adding integer values to the ASCII value for '0'. The resulting ASCII character was then written directly to the VGA character register.

To satisfy the project requirement regarding even-number handling, the application replaced all even digits with the ASCII character '0'. This produced the sequence: 0, 1, 0, 3, 0, 5, 0, 7, 0, 9

Each displayed character was rendered entirely by the hardware font pipeline described previously. The application itself did not perform any graphics operations or bitmap manipulation directly. Instead, the software simply updated the control registers while the hardware continuously generated VGA timing and pixel output in the background.

A software delay loop was used between character updates to slow visible display changes to approximately one-second intervals. Testing confirmed the correct operation of the complete software-to-hardware rendering path. The application successfully demonstrated:

- Wishbone memory-mapped communication
- Hardware-based font rendering
- ASCII character display
- Foreground and background color control
- Stable VGA timing generation
- Software-controlled display updates

Application 2

Application 2 was implemented as a graphics-mode VGA screensaver. Unlike Application 1, which used the text rendering path and font ROM, this application used the graphics mode of the VGA peripheral to animate a colored square across the screen.

The application communicates with the VGA peripheral using the same memory-mapped register interface as Application 1. At startup, the program writes to the mode register at base address 0x80001500 to select graphics mode. In this mode, the hardware ignores the character register and instead

renders a 32x32 square at the row and column stored in the coordinate register.

The C program maintains the square's current position using two software variables, row and col. It also maintains independent velocity values, row_step and col_step, which determine how many pixels the square moves vertically and horizontally each update. On each iteration of the main loop, the program packs the current row and column into the coordinate register format and writes the result to the VGA peripheral. The hardware then uses those coordinate values during VGA scanout to determine where the square should appear.

The application also controls the square's color through the VGA data register. In graphics mode, the hardware uses the foreground color fields of this register to determine the color of the rendered square. The application defines a small color palette in C using 4-bit red, green, and blue values. Whenever the square collides with a screen boundary, the program advances to the next color in the palette and writes the new RGB value to the data register.

Collision detection is handled entirely in software. After each position update, the application checks whether the square would move beyond the left, right, top, or bottom edge of the 640x480 display area. If a collision is detected, the position is clamped to the edge of the screen and the corresponding velocity component is negated. This produces the bouncing behavior. The program also varies the motion speed over time so that the animation is less repetitive than a fixed-path bouncing square.

A software delay loop is used to control the update rate of the animation. This delay determines how quickly new coordinates are written to the VGA peripheral. The VGA hardware continues generating synchronization signals and pixel output continuously while the software periodically updates only the control registers.

This application demonstrates a different use of the same VGA peripheral as Application 1. Instead of using the font ROM and character register, it uses the graphics rendering path, coordinate register, and color register to produce a simple real-time animation. The final result is a bouncing color-changing square that verifies software-controlled graphics output, dynamic coordinate updates, color control, and stable VGA rendering using the custom Wishbone peripheral.